Planejamento Lógico TaskTracker

Rafael Robson Nunes de Araújo

Bootcamp II | Desafio Cumulativo | Entrega Inicial (Fase 1)

1. Apresentação e Entendimento do Problema

1.1 Identificação

Campo	Dado
Nome completo	Rafael Robson Nunes de Araújo
Curso	Ciência de Dados e Machine Learning
Polo / Turma	Turma A - 0726 - Campus Virtual - Virtual - 2026/2
E-mail institucional	rafael.robson@sempreceub.com
Disciplina	Bootcamp II (EAD, Disciplina Virtual)
Entrega	Fase 1: Planejamento Lógico (10,0 pontos)

1.2 O problema a ser resolvido

Quem organiza as próprias tarefas na memória, em bilhetes soltos ou em conversas de aplicativo de mensagem convive com três falhas previsíveis. A primeira é a perda de informação: a tarefa existe, mas ninguém lembra do prazo, e ela só reaparece quando já está atrasada. A segunda é a ausência de prioridade explícita. Sem um critério registrado, a pessoa decide o que fazer pelo que está mais visível no momento, e não pelo que é mais urgente ou mais importante. A terceira é a falta de histórico: não existe nenhum registro do que foi concluído, então é impossível avaliar quanto trabalho realmente foi entregue em uma semana.

Em equipe o problema piora, porque cada pessoa mantém a própria lista em um formato diferente. Duas pessoas fazem a mesma tarefa, outra fica sem saber que era responsável por algo, e o prazo combinado verbalmente não tem prova. O ponto comum de todos esses casos é a ausência de um lugar único, com regras claras, onde a tarefa seja registrada de forma padronizada e permaneça registrada mesmo depois que o computador for desligado.

1.3 A solução proposta: TaskTracker

O TaskTracker é um sistema de gerenciamento de tarefas executado no terminal, escrito em Python, que transforma a lista informal em um registro estruturado e persistente. Ele resolve cada uma das falhas descritas acima com uma decisão concreta de projeto:

- Contra a perda de informação: toda tarefa é gravada em um arquivo de dados no disco. Fechar o programa não apaga nada; ao reabrir, a lista está exatamente como estava.
- Contra a falta de prioridade: prioridade é um campo obrigatório e restrito a três valores (Alta, Média, Baixa). Não existe tarefa sem prioridade definida, e a listagem padrão ordena por prioridade e depois por data limite.

- Contra a falta de histórico: a tarefa concluída não é apagada, muda de status. O sistema consegue responder quantas tarefas foram concluídas e quantas continuam pendentes.
- Contra dados inconsistentes: toda entrada passa por validação antes de ser aceita. Título vazio, prioridade inválida ou data mal formatada são recusados com uma mensagem que explica o erro, em vez de serem gravados e quebrarem o programa depois.

O usuário interage por um menu numerado com as operações de cadastrar, listar, concluir, excluir e sair. A entrega da Fase 1 é o desenho lógico completo dessas operações; a Fase 2 traduz este desenho em código Python versionado com Git; a Fase 3 empacota a aplicação em um container Docker.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de dados

São os dados que o usuário digita. As três primeiras colunas definem o contrato de cada campo; a última define o que o sistema faz quando o dado chega errado.

Campo	Tipo	Obrigatório	Exemplo	Validação aplicada
Opção do menu	Número	Sim	1	Precisa ser um dos números exibidos no menu; qualquer outro valor exibe aviso e repete o menu.
Título	Texto	Sim	Entregar relatório de vendas	Não pode ser vazio nem só espaços; limite de 100 caracteres.
Descrição	Texto	Não	Consolidar os números de agosto	Campo livre; se vazio, é gravado como texto vazio.
Prioridade	Texto	Sim	Alta	Só aceita Alta, Média ou Baixa, sem diferenciar maiúsculas de minúsculas.
Data limite	Data	Sim	20/09/2026	Formato DD/MM/AAAA, data existente no calendário e não anterior à data de hoje.
ID da tarefa	Número	Sim*	3	Usado ao concluir ou excluir; precisa corresponder a uma tarefa existente.
Confirmação	Texto	Sim*	S	Aceita S ou N; exigida antes de excluir uma tarefa.

** Obrigatório apenas dentro das operações que dependem do campo.*

2.2 Saídas de dados

São as respostas que o sistema devolve na tela. Toda operação termina com uma saída explícita; o usuário nunca fica sem retorno.

Saída	Conteúdo exibido
Menu principal	Lista numerada das operações disponíveis, exibida a cada volta do programa.
Confirmação de cadastro	Mensagem de sucesso informando o ID gerado para a nova tarefa.
Lista de tarefas	Tabela com ID, título, prioridade, data limite e status, ordenada por prioridade (Alta primeiro) e, dentro de cada prioridade, por data limite mais próxima.
Aviso de lista vazia	Mensagem informando que ainda não há tarefas cadastradas, em vez de uma tela em branco.
Resumo quantitativo	Total de tarefas, total de pendentes e total de concluídas, exibido no rodapé da listagem.
Destaque de atraso	Marcação visual nas tarefas pendentes cuja data limite já passou.
Mensagem de erro	Texto que nomeia o campo inválido e explica o formato esperado, sem encerrar o programa.
Confirmação de conclusão ou exclusão	Mensagem informando qual tarefa mudou de status ou foi removida.

2.3 Regras do sistema

São as condições que o sistema respeita em qualquer situação. Elas valem tanto para o algoritmo desta fase quanto para o código da Fase 2.

Regra	Descrição
R1	O sistema não aceita tarefas sem título. Título vazio ou composto apenas de espaços é recusado e o cadastro é interrompido.
R2	A prioridade só pode ser Alta, Média ou Baixa. Qualquer outro valor é recusado e o sistema pergunta novamente.
R3	A data limite precisa estar no formato DD/MM/AAAA, existir no calendário e não pode ser anterior à data atual.
R4	Cada tarefa recebe um ID numérico único, gerado pelo sistema, que nunca muda e nunca é reaproveitado após uma exclusão.
R5	Toda tarefa nasce com status "Pendente". A única transição permitida é de Pendente para Concluída.
R6	Concluir uma tarefa não a remove da base. Ela permanece registrada para efeito de histórico e de contagem.
R7	Exclusão exige confirmação explícita do usuário e só ocorre se o ID informado corresponder a uma tarefa existente.
R8	Toda alteração é gravada no arquivo de dados antes do sistema voltar ao menu, de modo que nada se perde ao encerrar o programa.
R9	Entrada inválida nunca derruba o programa: o sistema exibe a mensagem de erro e devolve o controle ao menu.

3. Receita Lógica do Sistema (Algoritmo em Português)

O passo a passo abaixo descreve o funcionamento completo do sistema em linguagem comum, como uma receita. Cada bloco corresponde a uma função que será escrita em Python na Fase 2.

3.1 Fluxo principal

1. Iniciar o programa.
2. Verificar se o arquivo de dados já existe. Se existir, carregar as tarefas gravadas para a memória; se não existir, começar com uma lista vazia.
3. Exibir o menu com as opções: 1 Cadastrar tarefa, 2 Visualizar tarefas, 3 Concluir tarefa, 4 Excluir tarefa, 5 Sair.
4. Ler a opção digitada pelo usuário.
5. Se a opção não estiver no menu, exibir aviso de opção inválida e voltar ao passo 3.
6. Executar a receita correspondente à opção escolhida.
7. Ao terminar a operação, voltar ao passo 3.
8. Se a opção for 5, gravar os dados, exibir mensagem de despedida e encerrar o programa.

3.2 Passo a passo do cadastro

1. Pedir ao usuário o título da tarefa.
2. Remover espaços sobrando no início e no fim do texto digitado.
3. Verificar se o título ficou vazio. Se ficou, exibir "O título não pode ficar em branco", cancelar o cadastro e voltar ao menu.
4. Pedir a descrição, deixando claro que o campo é opcional.
5. Pedir a prioridade e verificar se é Alta, Média ou Baixa. Se não for, exibir os valores aceitos e pedir novamente.
6. Pedir a data limite no formato DD/MM/AAAA.
7. Verificar se a data existe no calendário e se não é anterior a hoje. Se falhar em qualquer um dos dois testes, explicar o problema e pedir a data novamente.
8. Gerar o próximo ID: somar 1 ao maior ID já usado; se não houver nenhuma tarefa, o ID é 1.
9. Montar a tarefa com ID, título, descrição, prioridade, data limite e status "Pendente".
10. Acrescentar a tarefa à lista em memória.
11. Gravar a lista no arquivo de dados.
12. Exibir "Tarefa cadastrada com sucesso" junto com o ID gerado e voltar ao menu.

3.3 Passo a passo da visualização

1. Acessar a lista de tarefas carregada na memória.
2. Verificar se existe alguma tarefa.
3. Se a lista estiver vazia, avisar o usuário que ainda não há tarefas cadastradas e voltar ao menu.
4. Se houver tarefas, ordenar a lista: primeiro as de prioridade Alta, depois Média, depois Baixa; dentro de cada prioridade, a data limite mais próxima vem primeiro.
5. Exibir o cabeçalho das colunas: ID, Título, Prioridade, Data limite e Status.
6. Percorrer a lista e mostrar uma tarefa por linha na tela.

7. Para cada tarefa pendente cuja data limite já passou, marque a linha como atrasada.
8. Contar quantas tarefas existem no total, quantas estão pendentes e quantas estão concluídas.
9. Exibir esse resumo abaixo da lista e voltar ao menu.

3.4 Passo a passo da conclusão

1. Exibir a lista de tarefas para o usuário identificar o que quer concluir.
2. Pedir o ID da tarefa.
3. Procurar na lista a tarefa com aquele ID.
4. Se não encontrar, avisar que o ID não existe e voltar ao menu.
5. Se a tarefa já estiver concluída, informar que não há nada a alterar e voltar ao menu.
6. Trocar o status da tarefa de "Pendente" para "Concluída".
7. Gravar a lista no arquivo de dados.
8. Exibir a confirmação com o título da tarefa concluída e voltar ao menu.

3.5 Passo a passo da exclusão

1. Exibir a lista de tarefas.
2. Pedir o ID da tarefa a excluir.
3. Procurar a tarefa correspondente. Se não existir, avisar e voltar ao menu.
4. Mostrar o título da tarefa encontrada e perguntar "Confirma a exclusão? (S/N)".
5. Se a resposta for N, cancelar a operação e voltar ao menu sem alterar nada.
6. Se a resposta for S, remover a tarefa da lista.
7. Gravar a lista no arquivo de dados, sem reaproveitar o ID removido.
8. Exibir a confirmação da exclusão e voltar ao menu.

3.6 Passo a passo da persistência

Duas rotinas de apoio, usadas pelas receitas acima, garantem que os dados sobrevivam ao fechamento do programa.

Carregar:

1. Verificar se o arquivo de dados existe na pasta do projeto.
2. Se não existir, devolver uma lista vazia.
3. Se existir, ler o conteúdo e converter em lista de tarefas.
4. Se o arquivo estiver corrompido ou ilegível, avisar o usuário, trabalhar com lista vazia e não sobrescrever o arquivo original.

Gravar:

1. Converter a lista de tarefas da memória para o formato do arquivo.
2. Escrever o conteúdo no arquivo de dados, substituindo a versão anterior.
3. Executar essa gravação ao final de todo cadastro, conclusão e exclusão, antes de voltar ao menu.

4. Mapeamento Arquitetural para Git e GitHub (Preparação para a Fase 2)

4.1 Desenho da árvore do repositório

```
tasktracker-projeto/
├── README.md                    # Apresentação do projeto no GitHub
├── .gitignore                   # Arquivos ignorados pelo versionamento
├── requirements.txt             # Dependências do projeto
├── docs/
│   └── planejamento_logico.pdf  # Este documento da Fase 1
├── data/
│   └── tarefas.json             # Base de dados local (ignorada pelo Git)
├── src/
│   ├── main.py                  # Ponto de entrada: menu e fluxo principal
│   ├── tarefa.py                # Estrutura de uma tarefa e seus status
│   ├── repositorio.py           # Carregar e gravar o arquivo de dados
│   └── validacoes.py            # Regras R1 a R9 de validação de entrada
└── tests/
    └── test_validacoes.py       # Testes das regras de validação
```

4.2 Responsabilidade de cada arquivo

A separação abaixo existe para que cada arquivo tenha um motivo único para mudar. Alterar uma regra de validação não exige tocar no menu, e trocar o formato de armazenamento não exige reescrever a lógica das tarefas.

Arquivo	Responsabilidade
src/main.py	Exibe o menu, lê a opção do usuário e chama a operação correspondente. Não contém regras de negócio nem acesso a arquivo.
src/tarefa.py	Define o que é uma tarefa: campos, status possíveis e a transição de Pendente para Concluída.
src/repositorio.py	Única parte do sistema que lê e escreve no arquivo de dados. Isola a persistência do restante do código.
src/validacoes.py	Concentra as regras R1 a R9. Recebe um dado bruto e devolve o dado válido ou o motivo da recusa.
tests/	Verifica automaticamente se as validações continuam funcionando após cada alteração.
.gitignore	Impede o envio de arquivos temporários, da pasta __pycache__ e da base local data/tarefas.json.
README.md	Explica o que o projeto faz, como instalar e como executar.

4.3 Estratégia de versionamento planejada

- Repositório público no GitHub com o mesmo nome da pasta raiz do projeto.
- Branch main sempre em estado funcional; cada funcionalidade nova desenvolvida em uma branch própria, como feature/cadastro-tarefa ou feature/persistencia-json.
- Commits pequenos e descritivos, escritos no imperativo e indicando o efeito da mudança, por exemplo "adiciona validação de data limite".

- Integração das branches na main por pull request, para deixar registrado o histórico de cada decisão.

4.4 Plano de aprendizado para Git e GitHub

Na Fase 2 espero deixar de usar o Git apenas como um lugar de backup e passar a usá-lo como registro do raciocínio por trás do projeto. Quero entender na prática o que acontece em cada etapa do ciclo de trabalho: o que muda ao adicionar um arquivo, o que exatamente é fotografado em um commit e por que o histórico permite voltar a qualquer ponto anterior sem perder o que veio depois. Pretendo criar branches para desenvolver cada funcionalidade do TaskTracker de forma isolada, integrar essas branches na main e resolver os conflitos que aparecerem, já que conflito é consequência normal de trabalho paralelo e não um erro a ser evitado. Também quero aprender a usar o .gitignore com critério, para que a base de dados local e os arquivos temporários fiquem fora do repositório, e a escrever um README que faça alguém entender e executar o projeto sem precisar me perguntar nada. O objetivo final é que o histórico de commits conte a construção do sistema com a mesma clareza com que este documento descreve a lógica dele.

5. Link da Apresentação em Vídeo

Vídeo gravado no formato horizontal (16:9), com rosto e áudio, publicado no YouTube como Público ou Não Listado.

Item	Dado
Link do vídeo	[COLE AQUI O LINK DO SEU VÍDEO NO YOUTUBE]
Duração	5 minutos